

## ■ Lab 6 — Locals

### ■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Déclaré des **locals** pour centraliser et simplifier les expressions
- Référencé des locals avec `local`.
- Utilisé les locals pour les **tags communs** et les **noms de ressources**

### ■ Étape 1 — Créer les fichiers

```
cd labs/lab06-locals
```

#### ``backend.tf``

```
terraform {
  backend "s3" {
    bucket      = "tf-training-<votre-prenom>-982908300187"
    key         = "terraform.tfstate"
    region      = "eu-west-1"
    use_lockfile = true
    encrypt     = true
  }
}
```

■ ■ Remplacez `` par votre prénom en minuscules sans accent. Ex : `tf-training-alice-982908300187`

#### ``versions.tf``

```
terraform {
  required_version = ">= 1.15.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

#### ``provider.tf``

```
provider "aws" {  
  region = var.region  
}
```

### `variables.tf`

```
variable "username" {  
  description = "Votre prenom"  
  type        = string  
}  
  
variable "region" {  
  description = "Region AWS"  
  type        = string  
  default     = "eu-west-1"  
}  
  
variable "environment" {  
  description = "Environnement de déploiement"  
  type        = string  
  default     = "dev"  
  
  validation {  
    condition     = contains(["dev", "prod"], var.environment)  
    error_message = "L'environnement doit être dev ou prod."  
  }  
}
```

### `locals.tf`

```
locals {  
  # Préfixe commun pour nommer toutes les ressources  
  prefix = "lab06-${var.username}-${var.environment}"  
  
  # Tags appliqués à toutes les ressources  
  common_tags = {  
    Lab           = "lab6"  
    Username      = var.username  
    Environment   = var.environment  
    ManagedBy     = "Terraform"  
  }  
  
  # Type d'instance selon l'environnement  
  instance_type = var.environment == "prod" ? "t2.small" : "t2.micro"  
}
```

■ Les locals permettent d'assigner un nom à une expression pour éviter la répétition.

On les référence avec `local.` dans le reste du code.

### `main.tf`

```
resource "aws_security_group" "lab6_sg" {
  name          = "${local.prefix}-sg"
  description   = "Security group ${local.prefix}"

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-sg"
  })
}

resource "aws_instance" "lab6_instance" {
  ami              = "ami-0694d931cee176e7d"
  instance_type    = local.instance_type
  vpc_security_group_ids = [aws_security_group.lab6_sg.id]

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-instance"
  })
}
```

■ `merge()` est une fonction Terraform qui fusionne plusieurs maps. Ici on fusionne les `common_tags` avec le tag `Name` spécifique à chaque ressource.

### `outputs.tf`

```
output "instance_id" {
  description = "ID de l'instance EC2"
  value       = aws_instance.lab6_instance.id
}

output "instance_type_used" {
  description = "Type d'instance utilisé selon l'environnement"
  value       = local.instance_type
}

output "prefix_used" {
  description = "Préfixe utilisé pour nommer les ressources"
  value       = local.prefix
}
```

## ■ Étape 2 — Déployer et observer les locals

```
terraform init
# Déployer en dev
terraform apply -var="username=<votre-prenom>" -var="environment=dev"
```

Vérifiez le type d'instance utilisé :

```
terraform output instance_type_used
```

Résultat attendu : `t2.micro`

## ■ Étape 3 — Tester le local conditionnel

Redéployez en `prod` et observez le changement de `instance_type` :

```
terraform apply -var="username=<votre-prenom>" -var="environment=prod"
```

```
terraform output instance_type_used
```

Résultat attendu : `t2.small`

■ Le local `instance_type = var.environment == "prod" ? "t2.small" : "t2.micro"` est une **expression conditionnelle** — Terraform choisit automatiquement le bon type selon l'environnement.

## ■ Étape 4 — Utiliser terraform console

`terraform console` permet de tester les expressions et les locals de manière interactive :

```
terraform console -var="username=<votre-prenom>" -var="environment=dev"
```

Dans la console :

```
> local.prefix
"lab6-<votre-prenom>-dev"

> local.instance_type
"t2.micro"

> local.common_tags
{
  "Environment" = "dev"
  "Lab"         = "lab6"
  "ManagedBy"  = "Terraform"
  "Username"    = "<votre-prenom>"
}
```

Tapez `exit` pour quitter.

## ■ Étape 5 — Nettoyage

```
terraform destroy -var="username=<votre-prenom>" -var="environment=dev"
```

## ■ Validation du Lab

| # | Critère                                                              | Validé |
|---|----------------------------------------------------------------------|--------|
| 1 | `locals.tf` créé avec `prefix`, `common_tags` et `instance_type`     | ■      |
| 2 | `local.prefix` utilisé pour nommer toutes les ressources             | ■      |
| 3 | `local.common_tags` appliqué via `merge()` sur toutes les ressources | ■      |
| 4 | `terraform output instance_type_used` retourne `t2.micro` en dev     | ■      |
| 5 | `terraform output instance_type_used` retourne `t2.small` en prod    | ■      |
| 6 | `terraform console` affiche les valeurs des locals                   | ■      |
| 7 | `terraform destroy` supprime les ressources                          | ■      |

## ■■ Résolution des problèmes courants

| Problème              | Cause                       | Solution                                |
|-----------------------|-----------------------------|-----------------------------------------|
| `local.xxx undefined` | Faute de frappe dans le nom | Vérifier l'orthographe dans `locals.tf` |

| Problème                      | Cause               | Solution                                      |
|-------------------------------|---------------------|-----------------------------------------------|
| <code>`merge()`</code> échoue | Types incompatibles | Vérifier que les deux arguments sont des maps |

■ **\*\*Fin du Lab 6 — Votre code est centralisé et sans répétition !\*\***

*Le Lab 7 introduit **`count`** et **`count.index`** pour itérer sur les ressources.*